

User Guide: Syntax Binding XML-to-Hierarchical-CSV Conversion

1. Working Directory

Run commands from the **UADC_PoC** directory:

```
cd UADC_PoC
```

All paths below are relative to this directory unless they begin with `../`.

Set the Python command for the local Windows environment:

```
$python = 'python'
```

2. Main Script

Use:

```
src/syntax_binding.py
```

This script converts XML to dimension-based hierarchical CSV using a syntax binding CSV. It also supports reverse conversion from hierarchical CSV back to XML with **--reverse**.

For implementation details, including XPath parent-context recursion, Semantic Path resolution, **dInvoice** and **dInvoiceLine** assignment, and function-level data flow, see **../README_SCRIPT_PROCESSING.md**.

3. Convert the PoC OpenPeppol Sample

Run:

```
& $python .\src\syntax_binding.py `
.\samples\input\openpeppol_ubl_invoice_minimal.xml `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
--metadata-output .\out\phase1\openpeppol_ubl_invoice_minimal.json `
--taxonomy-base .\out\taxonomy `
-o .\out\phase1\openpeppol_ubl_invoice_minimal.csv
```

Output:

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
out/phase1/openpeppol_ubl_invoice_minimal.json
```

This command uses the LHM-derived columns embedded in the syntax binding CSV to determine the output dimensions and fact columns. It also creates JSON metadata that relates the CSV columns to the generated taxonomy.

The converter treats **lhm_level** as the effective hierarchy for Structured CSV. BG rows with blank **lhm_level**, such as non-repeating Seller or Buyer groups, are semantic grouping nodes only; their BT values are written in the nearest ancestor dimension row.

4. Convert the Revised Package Sample

Run:

```
& $python .\src\syntax_binding.py `
  ..\syntax_binding_revised_package\invoice.xml `
  -b ..\syntax_binding_revised_package\bindings.csv `
  --template-csv ..\syntax_binding_revised_package\invoice.csv `
  --metadata-output .\out\phase1\package_invoice_hierarchical.json `
  -o .\out\phase1\package_invoice_hierarchical.csv
```

Output:

```
out/phase1/package_invoice_hierarchical.csv
out/phase1/package_invoice_hierarchical.json
```

This command uses the package template CSV to preserve the expected output column order.

5. Command Options

Basic form:

```
& $python .\src\syntax_binding.py XML_FILE `
  -b BINDING_CSV `
  -o OUTPUT_CSV
```

Options:

- **--template-csv**: use a template CSV header to define output column order.

- **--metadata-output**: set the JSON metadata output path. Default is the output CSV path with **.json** extension.
- **--taxonomy-base**: set the generated taxonomy directory referenced by JSON metadata. Default is **out/taxonomy**.
- **--reverse**: convert hierarchical CSV back to XML.
- **--ubl-schema-root**: local directory containing extracted UBL XSD files. In reverse mode, child element order is derived from XSD **xs:sequence** declarations.
- **--ubl-schema-url**: UBL Invoice XSD URL. In reverse mode, imported and included schemas are followed from this URL and child element order is derived from XSD **xs:sequence** declarations.
- **--drop-empty-columns**: remove output columns that have no values.
- **--d-invoice**: set the **dInvoice** value. Default is **1**.
- **-e, --encoding**: CSV encoding. Default is **utf-8-sig**.

6. Reverse Hierarchical CSV to XML

First create or confirm the hierarchical CSV:

```
& $python .\src\syntax_binding.py `
.\samples\input\openpeppol_ubl_invoice_minimal.xml `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
--metadata-output .\out\phase1\openpeppol_ubl_invoice_minimal.json `
-o .\out\phase1\openpeppol_ubl_invoice_minimal.csv
```

Then reverse it to XML:

```
& $python .\src\syntax_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
--reverse `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
-o .\out\reverse\en16931_reverse_invoice.xml
```

Output:

```
out/reverse/en16931_reverse_invoice.xml
```

To derive child element order from a local UBL schema package, add **--ubl-schema-root**:

```
& $python .\src\syntax_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
--reverse `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
```

```
--ubl-schema-root .\out\cache\UBL-2.1\xsd `
-o .\out\reverse\en16931_reverse_invoice.xml
```

To derive child element order from an online UBL Invoice schema entry point, use **--ubl-schema-url**. The converter follows imported and included schemas from that URL.

The reverse XML is generated from bound CSV values. It is intended for round-trip verification and may not reproduce unbound XML content. When the LHM has **syntax_sequence** values, reverse output uses them to follow UBL schema order.

Amount **currencyID** attributes are derived during reverse conversion:

- **DocumentCurrencyCode** is used for ordinary invoice amount elements.
- **TaxAccountingCurrencyCode** is used for the tax accounting currency TaxTotal branch.

Forward conversion also uses those currency terms to distinguish TaxTotal branches. In **Allowance-example.xml**, BT-110 selects the TaxAmount whose **currencyID** matches **DocumentCurrencyCode** (1225.00 EUR), while BT-111 selects the TaxAmount whose **currencyID** matches **TaxAccountingCurrencyCode** (9324.00 SEK).

Some semantic children are stored outside their repeated UBL syntax context. BT-90 belongs semantically to payment instructions/direct debit, but its UBL XPath is below

AccountingSupplierParty. During reverse conversion, an absolute binding XPath that is not contained by the current repeated XPath is written from the document root. It must not produce a nested **Invoice** below **PaymentMeans**.

7. Generate Test Artifacts with Metadata

Round-trip test artifacts are built under:

```
tests/roundtrip/
```

Run:

```
& $python .\tools\build_roundtrip_test_artifacts.py
```

The generated layout is:

```
tests/roundtrip/<dataset>/
  source_xml/
  structured_csv/
  metadata_json/
  roundtrip_xml/
```

Meaning:

- **source_xml**: source UBL Invoice XML.
- **structured_csv**: hierarchical structured CSV.
- **metadata_json**: JSON metadata relating the CSV columns to the taxonomy.
- **roundtrip_xml**: XML regenerated from the structured CSV.

The script uses **--metadata-output** to place metadata JSON in **metadata_json/**.

8. Input Binding CSV

The binding CSV should contain:

```
semantic_path, xpath
```

Example:

```
$.invoice.invoiceNumber,/Invoice/cbc:ID
$.invoice.invoiceIssueDate,/Invoice/cbc:IssueDate
```

Compatible column aliases are also accepted:

```
path
source_xpath
xml_path
```

9. Output CSV Layout

The hierarchical CSV uses:

- **dInvoice** for the invoice root;
- **d*** columns for repeated BG dimensions;
- **fact** columns for BT values.

When the LHM is used, dimension columns are placed first and BT columns follow.

Example output pattern:

```
dInvoice,dVatBreakdown,dInvoiceLine,InvoiceNumber,InvoiceIssueDate,...
1,,,INV-2026-0001,2026-07-06,...
1,1,,,,...
1,1,,,,...
```

10. JSON Metadata Layout

The metadata file is an xBRL-CSV metadata document. It has these main sections:

- **documentInfo:** xBRL-CSV document type and taxonomy references.
- **tables:** the generated structured CSV table URL.
- **tableTemplates:** table-level dimensions and fact concept mappings.

Check that important columns are mapped as expected:

```
tableTemplates.structured.dimensions["plt:d_en16931_Invoice"] ->
"$dInvoice"
tableTemplates.structured.columns.InvoiceNumber.dimensions.concept ->
"en16931:InvoiceNumber"
tableTemplates.structured.columns.DocumentCurrencyCode.dimensions.concept
-> "en16931:DocumentCurrencyCode"
```

11. Run Regression Checks

Run the LHM-driven OpenPeppol sample conversion check:

```
& $python .\tests\test_lhm_hierarchical_csv_layout.py
```

Run the revised package conversion check:

```
& $python .\tests\test_syntax_binding.py
```

Run the OpenPeppol structured conversion check:

```
& $python .\tests\test_openpeppol_invoice_conversion.py
```

Run the reverse conversion round-trip check:

```
& $python .\tests\test_syntax_binding_reverse.py
```

Run the round-trip artifact and metadata check:

```
& $python .\tests\test_roundtrip_artifacts.py
```

Validate the generated xBRL-CSV metadata with Arelle:

```
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```

Validate the regenerated round-trip XML against the UBL 2.1 Invoice schema:

```
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

12. Troubleshooting

No usable bindings found

Check that the binding CSV contains **semantic_path** and **xpath** values.

Missing output columns

If **--drop-empty-columns** is used, columns with no generated values are removed. Run without this option when a stable full header is required.

Metadata JSON cannot locate the taxonomy

The converter must write a non-empty **documentInfo.taxonomy** array. Generate the taxonomy first:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

Or pass **--taxonomy-base** with the directory containing **plt/en16931-oim-*.xsd**. If this schema is missing, metadata generation fails instead of writing an empty taxonomy list.

Repeating rows are missing

Check that:

- the BG row in the LHM has repeating multiplicity such as **0..*** or **1..***;
- the BG row has an XPath that points to the repeated XML context;
- the syntax binding CSV contains bindings under that BG semantic path.

Values are written to the invoice root row instead of a dimension row

Check that the syntax binding CSV contains the LHM-derived **type**, **multiplicity**, **semantic_path**, and **structured_csv_column** columns. The converter resolves each BT semantic path to its nearest repeated C-row dimension from the binding table itself.

Package sample line rows are empty

Template columns alone do not create rows. Invoice line rows require invoice line bindings.

Reverse XML does not match the original file order

Reverse XML is reconstructed from binding rows and hierarchical CSV values. Populate LHM **syntax_sequence** from the UBL schema when XML element order must be checked. Unbound XML content still cannot be reproduced until it is represented by a binding or fixed-value rule.